

1. Protocolo

Tecnológico Nacional de México

Instituto Tecnológico de León, Campus 1

Carrera: Ingeniería en Sistemas Computacionales

Materia: Estructura de Datos (Grupo B)

Horario: Lunes y miércoles de 8:45 a 10:25 hrs. y
Viernes de 8:45 a 9:35 hrs.

Alumno: Marlene Inés Moreno Velázquez

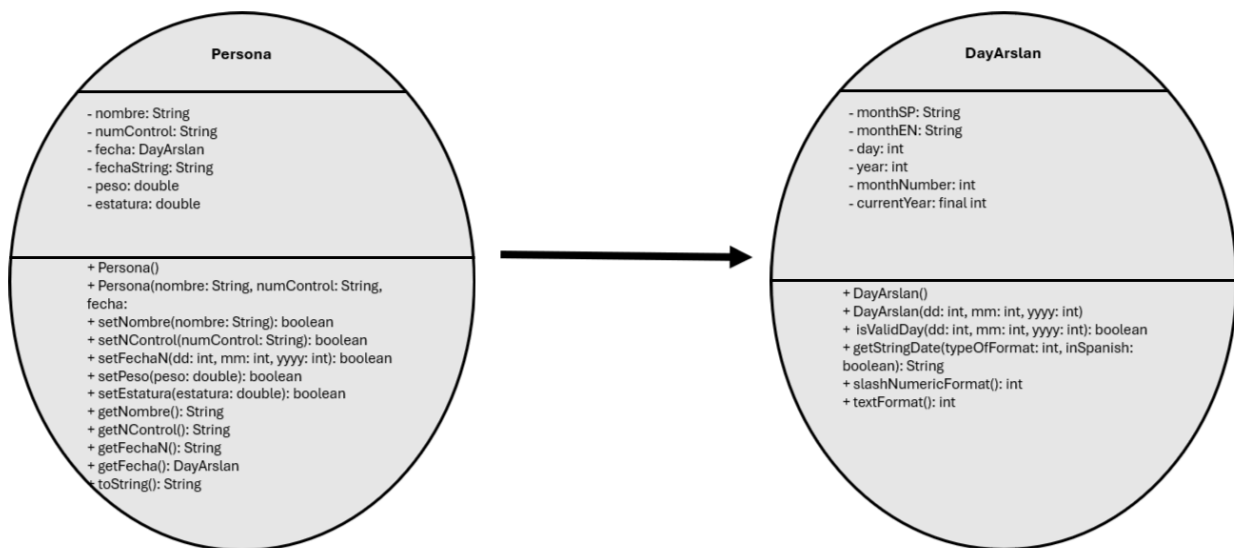
Número de lista: 12

Fecha: 25 de febrero de 2026.



Ejercicio #4: Clase Encapsulada “Persona”

2. Diagrama UML



3. Algoritmo

----- ALGORITMO PRUEBA PERSONA (CAPTURA DE DATOS) -----

1. meta():

 Escribir("""

 Programa que permita capturar los datos de la ficha de cada estudiante para así almacenar la información en archivo o arreglo y poder utilizarla para la búsqueda secuencial de cada uno""");

```

2. data():
    Persona[] grupo = nuevo Persona[25];
    cuenta = 0; stop = 0;

    terminar mientras (cuenta < grupo.length y stop != 1) empezar
        grupo[cuenta] = nuevo Persona();

        llamar a solicitarNombre(grupo, cuenta);
        llamar a solicitarNumControl(grupo, cuenta);
        llamar a solicitarFechaNacimiento(grupo, cuenta);
        llamar a solicitarEstatura(grupo, cuenta);
        llamar a solicitarPeso(grupo, cuenta);

        cuenta = cuenta + 1;

        Escribir("¿Desea continuar registrando más personas?");
        stop = ?;
    terminar mientras

2.1 solicitarNombre(Persona grupo, int id):
    correct = falso;

    terminar mientras (!correct) empezar
        Escribir("Escriba el nombre completo de la persona");
        nombre = ?;

        correct = grupo[id].setNombre(nombre);

        si (!correct) empezar
            Escribir("Asegurese de ingresar un nombre valido");
        terminar si
    terminar mientras

2.2 solicitarNumControl(Persona grupo, int id):
    correct = falso;

    terminar mientras (!correct) empezar
        Escribir("Escriba el numero de control de la persona");
        numControl = ?;

        correct = grupo[id].setNControl(numControl);

        si (!correct) empezar
            Escribir("Asegúrese de escribir un número de control valido.
                (8 dígitos con formato válido)");
        terminar si
    terminar mientras

```

2.3 solicitarFechaNacimiento(Persona grupo, int id):

correct = falso;

terminar mientras (!correct) empezar

intentar empezar

 Escribir("Ingrese el dia de nacimiento");

 dia = ?;

 Escribir("Ingrese el numero de mes de nacimiento");

 mes = ?;

 Escribir("Ingrese el año de nacimiento");

 year = ?;

 correct = grupo[id].setFecha(dia, mes, year);

 si (!correct) empezar

 Escribir("Asegúrese de escribir correctamente la fecha");

 terminar si

capturar error empezar

 Escribir("Asegurese de usar solo numeros.");

terminar intentar

terminar mientras

2.4 solicitarEstatura(Persona grupo, int id):

correct = falso;

terminar mientras (!correct) empezar

intentar empezar

 Escribir("Deme la estatura de la persona");

 estatura = ?;

 correct = grupo[id].setEstatura(estatura);

 si (!correct) empezar

 Escribir("Asegúrese de escribir correctamente la estatura");

 terminar si

capturar error empezar

 Escribir("Asegurese de usar un numero decimal valido.");

terminar intentar

terminar mientras

2.5 solicitarPeso(Persona grupo, int id):

correct = falso;

terminar mientras (!correct) empezar

intentar empezar

 Escribir("Deme el peso de la persona");

 peso = ?;

 correct = grupo[id].setPeso(peso);

 si (!correct) empezar

 Escribir("Asegúrese de escribir correctamente el peso");

 terminar si

capturar error empezar

 Escribir("Asegurese de usar un numero decimal valido.");

terminar intentar

terminar mientras

2.6 archivo():

```
file = new JFileChooser();

file.setDialogTitle("Seleccione un archivo txt");
resultado = file.showOpenDialog(nulo);
si (resultado == file.APPROVE_OPTION) empezar
    archivo = file.getSelectedFile();
    intentar empezar
        terminar mientras (linea = leerLinea() != null
                            and cuenta < grupo.length) empezar
            String[] datos = linea.split(",");

            si (datos.length == 5) empezar
                nombre = datos[0].trim();
                numControl = datos[1].trim();
                fecha = datos[2].trim();
                peso = datos[3].trim();
                estatura = datos[4].trim();
                grupo[cuenta] = new Persona(nombre, numControl, fecha,
                peso, estatura);
                cuenta = cuenta + 1;
            de otro modo empezar
                Escribir("Formato incorrecto en la linea: " + linea);
            terminar si
        terminar mientras
        Escribir("Se ha registrado correctamente desde archivo");
    capturar error empezar
        Escribir("Error al leer el archivo");
    terminar intentar
de otro modo empezar
    Escribir("No se selecciono ningun archivo");
terminar si
```

3. buscar(): // PROCESO

```
i = 0;
encontrado = falso;

Escribir("Ingrese el nombre a buscar");
nombreb = ?;

si (nombreb es nulo) regresar;

terminar mientras (i < cuenta y !encontrado) empezar
    si (grupo[i].getNombre() == nombreb) empezar
        encontrado = verdadero;
        Escribir("Persona encontrada");
        Escribir("Comparaciones: " + (i + 1));
    de otro modo empezar
```

```
        i = i + 1;
    terminar si
terminar mientras
```

```
    posicion = i;
```

4. imprimir(): // RESULTADOS

```
    i = 0; resultado = "";
```

```
    si (cuenta == 0) empezar
        Escribir("No hay registros");
        regresar;
```

```
    terminar si
```

```
    terminar mientras (i < cuenta) empezar
```

```
        resultado = resultado + "Persona " + (i+1);
```

```
        resultado = resultado + grupo[i];
```

```
        i = i + 1;
```

```
    terminar mientras
```

```
    Escribir("Lista completa:");
```

```
    Escribir(resultado);
```

5. main(): // NAVEGABILIDAD

```
    llamar a meta();
```

```
    opcion = menu();
```

```
    terminar mientras (opcion != -1 y opcion != 4) empezar
```

```
        segun (opcion) empezar
```

```
            caso 0:
```

```
                llamar a data();
```

```
                romper;
```

```
            caso 1:
```

```
                llamar a archivo();
```

```
                romper;
```

```
            caso 2:
```

```
                llamar a buscar();
```

```
                romper;
```

```
            caso 3:
```

```
                llamar a imprimir();
```

```
                romper;
```

```
        terminar segun
```

```
        opcion = menu();
```

```
    terminar mientras
```

5.1 menu(): int

```
    Escribir("0. Introducir datos");
```

```
    Escribir("1. Leer archivo");
```

```
    Escribir("2. Buscar persona");
```

```
    Escribir("3. Imprimir lista");
```

```
    Escribir("4. Salir");
```

```
opcion = ?;  
regresar opcion;
```

4. Código Clase Encapsulada Persona

```
import tools.DayArslan;  
  
public class Persona {  
    private String nombre;  
    private String numControl;  
    private DayArslan fecha;  
    private String fechaString = "";  
    private double peso, estatura;  
  
    public Persona(){  
        this.nombre = "";  
        this.numControl = "";  
        this.fecha = null;  
        this.peso = 0.0;  
        this.estatura = 0.0;  
    }  
  
    public Persona(String nombre, String numControl,  
        DayArslan fecha, double peso, double estatura){  
        this.nombre = nombre;  
        this.numControl = numControl;  
        this.fecha = fecha;  
        this.peso = peso;  
        this.estatura = estatura;  
    }  
  
    public Persona(String nombre, String numControl, String fecha,  
        double peso, double estatura){  
        this.nombre = nombre;  
        this.numControl = numControl;  
        this.fechaString = fecha;  
        this.peso = peso;  
        this.estatura = estatura;  
    }  
  
    public boolean setNombre(String nombre){  
        if(nombre == null || nombre.trim().isEmpty() || nombre.isBlank())  
            return false;  
        else if(!nombre.matches("[A-Za-zÁÉÍÓÚáéíóúñÑ ]+"))  
            return false;  
        else if(nombre.length() < 2 || nombre.length() > 50)  
            return false;  
    }  
}
```

```

        else{
            this.nombre = nombre.trim();
            return true;
        }
    }

    public boolean setNControl(String numControl){
        if(numControl == null || numControl.isBlank())
            return false;
        if(numControl.length() != 8)
            return false;
        if(!numControl.matches("\\d{8}"))
            return false;
        if(!numControl.substring(2, 4).equals("24"))
            return false;

        int firsttwo = Integer.parseInt(numControl.substring(0, 2));

        if(firsttwo < 18 || firsttwo > 25) return false;
        this.numControl = numControl;
        return true;
    }

    public boolean setFecha(int dd, int mm, int yyyy){
        DayArslan temp = new DayArslan(dd, mm, yyyy);
        if (!temp.isValidDay(dd, mm, yyyy)) return false;
        else {
            this.fecha = temp;
            return true;
        }
    }

    public boolean setPeso(double peso){
        if(peso <= 20 || peso > 250) return false;
        else{
            this.peso = peso;
            return true;
        }
    }

    public boolean setEstatura(double estatura){
        if(estatura <= 0.4 || estatura > 2.5) return false;
        else{
            this.estatura = estatura;
            return true;
        }
    }
}

```

```

@Override
public String toString(){
    if(fechaString.isEmpty() || fechaString.isBlank())
        return "\tNombre: " + nombre
            + "\n\tNumero de control: " + numControl
            + "\n\tFecha de Nacimiento: "
            + fecha.getStringDate(fecha.slashNumericFormat(), true)
            + "\n\tPeso en Kg: " + peso + "kg"
            + "\n\tEstatura en mts: " + estatura + "m";
    else return "\tNombre: " + nombre
        + "\n\tNumero de control: " + numControl
        + "\n\tFecha de Nacimiento: " + fechaString
        + "\n\tPeso en Kg: " + peso + "kg"
        + "\n\tEstatura en mts: " + estatura + "m";
}

public String getNombre(){return nombre;}

public String getNControl(){return numControl;}

public String getFechaN() {
    return fecha.getStringDate(fecha.slashNumericFormat(), true);
}

public double getPeso(){return peso;}

public double getEstatura(){return estatura;}

public DayArslan getFecha() {
    return fecha;
}

public void setFecha(DayArslan fecha) {
    this.fecha = fecha;
}
}

```

5. Código Clase Encapsulada DayArslan (recurso de autoría propia utilizado en la clase Persona)

```

package tools;

public class DayArslan{
    private String monthSP, monthEN;
    private int day, year, monthNumber;
    private final int currentYear = 2026;

    public DayArslan(){}
}

```



```

public DayArslan(int dd, int mm, int yyyy){
    this.day = dd;
    this.monthNumber = mm;
    this.year = yyyy;

    switch(monthNumber){
        case 1 -> { monthEN = "January"; monthSP = "Enero";}
        case 2 -> { monthEN = "February"; monthSP = "Febrero";}
        case 3 -> { monthEN = "March"; monthSP = "Marzo";}
        case 4 -> { monthEN = "April"; monthSP = "Abril";}
        case 5 -> { monthEN = "May"; monthSP = "Mayo";}
        case 6 -> { monthEN = "June"; monthSP = "Junio";}
        case 7 -> { monthEN = "July"; monthSP = "Julio";}
        case 8 -> { monthEN = "August"; monthSP = "Agosto";}
        case 9 -> { monthEN = "September"; monthSP = "Septiembre";}
        case 10 -> {monthEN = "October"; monthSP = "Octubre";}
        case 11 -> {monthEN = "November"; monthSP = "Noviembre";}
        case 12 -> {monthEN = "December"; monthSP = "Diciembre";}
    }
}

public boolean isValidDay(int dd, int mm, int yyyy) {
    boolean esBisiesto = (yyyy % 4 == 0 && yyyy % 100 != 0) ||
        (yyyy % 400 == 0);

    int diasEnMes;

    if (yyyy < 1960 || yyyy > this.currentYear) return false;
    if (mm < 1 || mm > 12) return false;
    switch (mm) {
        case 1, 3, 5, 7, 8, 10, 12 -> diasEnMes = 31;
        case 4, 6, 9, 11 -> diasEnMes = 30;
        case 2 -> diasEnMes = esBisiesto ? 29 : 28;
        default -> {return false;}
    }
    return dd >= 1 && dd <= diasEnMes;
}

public String getStringDate(int typeOfFormat, boolean inSpanish){
    String date = "", days, months, years;

    if(inSpanish){
        switch(typeOfFormat){
            case 1:
                if(this.day < 10) days = "0" + this.day;
                else days = Integer.toString(this.day);
                if(this.monthNumber < 10) months = "0" + this.monthNumber;
                else months = Integer.toString(this.monthNumber);
                years = Integer.toString(this.year);
                date = days + "/" + months + "/" + years; break;

```

```

        case 2:
            days = Integer.toString(this.day);
            months = this.monthSP;
            years = Integer.toString(this.year);
            date = days + " de " + months + " del " + years; break;
    }
} else{
    switch(typeOfFormat){
        case 1:
            if(this.day < 10) days = "0" + this.day;
            else days = Integer.toString(this.day);
            if(this.monthNumber < 10) months = "0" + this.monthNumber;
            else months = Integer.toString(this.monthNumber);
            years = Integer.toString(this.year);
            date = months + "/" + days + "/" + years; break;
        case 2:
            String ord;
            days = Integer.toString(this.day);
            if(day == 1 || day == 11 || day == 21 || day == 31)
                ord = "st";
            else if(day == 2 || day == 12 || day == 22)
                ord = "nd";
            else if(day == 3 || day == 13 || day == 23)
                ord = "rd";
            else ord = "th";
            months = this.monthEN;
            years = Integer.toString(this.year);
            date = months + ", " + days + ord + ", " + years; break;
    }
}
return date;
}

public int slashNumericFormat(){return 1;}

public int textFormat(){return 2;}
}

```

6. Código Clase de Prueba

```

import java.awt.HeadlessException;
import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import javax.swing.JFileChooser;
import tools.EntryPrompt;

```

```

public class Grupo {
    int cuenta = 0, stop, posicion, opcion;
    String nombreb, nombre, numControl;
    boolean encontrado;
    double estatura, peso;
    Persona[] grupo = new Persona[25]; EntryPrompt into = new EntryPrompt();

    void meta() {
        String mensaje = ""
            Registro de personas mediante teclado y lectura de archivo.
            Usted puede registrar hasta un máximo de 25 personas,
            buscar a una persona en específico mediante su nombre e
            imprimir sus datos o la lista completa.
            "";

        into.outPlain(mensaje, "ALGORITMO SECUENCIAL");
    }

    void entrada() { // DATA
        String opciones[] = {"Continuar", "Terminar"}; stop = 0;

        while (cuenta < grupo.length && stop != 1) {
            grupo[cuenta] = new Persona();

            solicitarNombre(grupo, cuenta);
            solicitarNumControl(grupo, cuenta);
            solicitarFechaNacimiento(grupo, cuenta);
            solicitarEstatura(grupo, cuenta);
            solicitarPeso(grupo, cuenta);

            cuenta++;
            stop = into.entryOption("¿Desea continuar registrando?",
                "REGISTRO DE DATOS", opciones);
        }
    }

    void solicitarNombre(Persona grupo[], int id){
        boolean correct = false; String error = "Campo OBLIGATORIO.\n";

        while (!correct) { //Nombre
            nombre = into.entryString("NOMBRE COMPLETO",
                "Escriba el nombre completo de la persona #"+(id+1));
            correct = grupo[id].setNombre(nombre);

            if (!correct) into.outError(error+
                "Asegúrese de ingresar un nombre válido.", "NOMBRE INCORRECTO");
        }
    }
}

```

```

void solicitarNumControl(Persona grupo[], int id){
    boolean correct = false;

    while (!correct) { //Numero de control
        numControl = into.entryString("NUMERO DE CONTROL",
            "Deme el número de control de la persona #"+(id+1));
        correct = grupo[id].setNControl(numControl);

        if (!correct) {
            String m = ""
                Campo OBLIGATORIO.
                Asegúrese de escribir correctamente el número de
                control.
                Requisitos:
                - Longitud de 8 dígitos
                - Inicia con el año de matriculación (ej. 2025 -> 25)
                - Seguido de la región del instituto -> 24
                - Termina con 4 dígitos de matrícula ####
                "";

            into.outError(m, "NUMERO DE CONTROL INCORRECTO");
        }
    }
}

```

```

void solicitarFechaNacimiento(Persona grupo[], int id){
    boolean correct = false;

    while (!correct) { //Fecha de nacimiento
        try {
            String title = "FECHA DE NACIMIENTO";
            int dia = into.entryInt(0, title,
                "Ingrese el día de nacimiento.");
            int mes = into.entryInt(0, title,
                "Ingrese el numero de mes de nacimiento.");
            int year = into.entryInt(0, title,
                "Ingrese el año de nacimiento.");
            correct = grupo[id].setFecha(dia, mes, year);

            if (!correct) into.outError("Asegúrese de introducir "
                +"correctamente la fecha utilizando números.",
                title+" INCORRECTA");
        } catch (NumberFormatException e) {
            into.outAdv("Asegúrese de introducir correctamente la fecha.
                \nUtilice ÚNICAMENTE números.", "FECHA INVÁLIDA");
            correct = false;
        }
    }
}

```

```

void solicitarEstatura(Persona grupo[], int id){
    boolean correct = false;
    while (!correct) { //Estatura
        try {
            String title = "ESTATURA EN METROS";
            estatura = into.entryDouble(0, title,
                "Deme la estatura de la persona #" + (id + 1));
            correct = grupo[id].setEstatura(estatura);

            if (!correct) into.outError("Asegúrese de introducir "
                + "correctamente la estatura.", title
                + " INCORRECTA");
        } catch (NumberFormatException e) {
            correct = false;

            into.outAdv("Campo OBLIGATORIO.\nAsegúrese de ingresar un "
                + "número decimal válido.", "ESTATURA NO VÁLIDA");
        }
    }
}

void solicitarPeso(Persona grupo[], int id){
    boolean correct = false;

    while (!correct) { //Peso
        try {
            String title = "PESO EN KILOGRAMOS";
            peso = into.entryDouble(0, title,
                "Deme el peso de la persona #" + (id + 1));
            correct = grupo[id].setPeso(peso);

            if (!correct) {
                into.outError("Campo OBLIGATORIO.\n"
                    + "Asegúrese de ingresar correctamente el peso.",
                    title + " INCORRECTO");
            }
        } catch (HeadlessException | NumberFormatException e) {
            correct = false;

            into.outAdv("Campo OBLIGATORIO.\nAsegúrese de ingresar un "
                + " número decimal válido.", "PESO NO VÁLIDO");
        }
    }
}

```

// 2.2 DATA - Archivo

```

void archivo() { //Abrir archivo desde navegador de archivos
    JFileChooser fileC = new JFileChooser();
    fileC.setDialogTitle("Seleccione un archivo de texto (.txt)");
}

```

```

int resultado = fileC.showOpenDialog(null);

if (resultado == JFileChooser.APPROVE_OPTION) {
    File archivo = fileC.getSelectedFile();

    try (BufferedReader br = new BufferedReader(
        new FileReader(archivo))) {
        String linea;

        while ((linea = br.readLine()) != null &&
            cuenta < grupo.length) {
            String[] datos = linea.split(",");

            if (datos.length == 5) {
                String nombre = datos[0].trim();
                String numControl = datos[1].trim();
                String fecha = datos[2].trim();
                String p = datos[3].trim();
                String es = datos[4].trim();
                double peso = Double.parseDouble(p);
                double estatura = Double.parseDouble(es);
                grupo[cuenta] = new Persona(nombre, numControl,
                    fecha, peso, estatura);

                cuenta++;
            } else into.outError("Formato incorrecto en la línea: "
                +linea, "ERROR");
        }
        into.outInfo("Se han registrado "+cuenta
            +" personas desde el archivo.", "LECTURA EXITOSA");
    } catch (Exception e) {
        into.outAdv("Error al leer el archivo: " + e.getMessage(),
            "ERROR");
    }
} else into.outError("No se seleccionó ningún archivo.",
    "ADVERTENCIA");
}

//3. PROCESO
void buscar() {
    int i = 0; encontrado = false;
    nombreb = into.entryString("BUSCAR PERSONA", "Deme el nombre de la "
        +"persona que desea buscar");

    if (nombreb.isBlank() || nombreb.isEmpty()) return;
    while (i < cuenta && !encontrado) {
        if (grupo[i].getNombre().equalsIgnoreCase(nombreb)) {
            into.outInfo("Persona encontrada.\nNúmero de comparaciones "
                +"realizadas: "+(i+1),"BÚSQUEDA EXITOSA");
            encontrado=true;
        }
        else i++;
    }
}

```

```

        posicion = i;
        if (encontrado)
            into.outPlain(grupo[posicion].toString(), "PERSONA ENCONTRADA");
        else into.outError("No se encontró en el grupo.", "RESULTADO");
    }
//4. RESULTADOS
void imprimir() {
    int i = 0; String resultado = "";

    if (cuenta == 0) {
        into.outError("No hay personas registradas.", "SIN REGISTROS");
        return;
    }
    while (i < cuenta) resultado += "\nPersona " + (i + 1) + "\n"
        + grupo[i].toString(); i++;

    System.out.println(
        "-----[LISTA DE TODAS LAS PERSONAS REGISTRADAS]-----\n"
        + resultado); into.outPlain("Revise la lista impresa en la "
        + "terminal de ejecución.", "IMPRESIÓN COMPLETA");
}
//5. NAVEGABILIDAD
int menu() {
    String mensajeOpciones = ""
        Usted puede:
        1. Introducir datos manualmente
        2. Leer datos desde un archivo de texto
        3. Buscar una persona por nombre
        4. Imprimir toda la lista
        5. Salir del programa

        Elija la opción que desee...
        "";
    String opciones[] = {"Introducir datos", "Leer datos de archivo",
        "Buscar persona", "Imprimir lista", "SALIR"};

    opcion = into.entryOption(mensajeOpciones, "MENÚ PRINCIPAL",
opciones);
    return opcion;
}

public static void main(String[] args) { // NAVEGABILIDAD
    Grupo mn = new Grupo(); mn.meta();
    int sw = mn.menu();

    while (sw != -1 && sw != 4) {
        switch (sw) {
            case 0 -> mn.entrada();
            case 1 -> mn.archivo();
            case 2 -> mn.buscar();

```

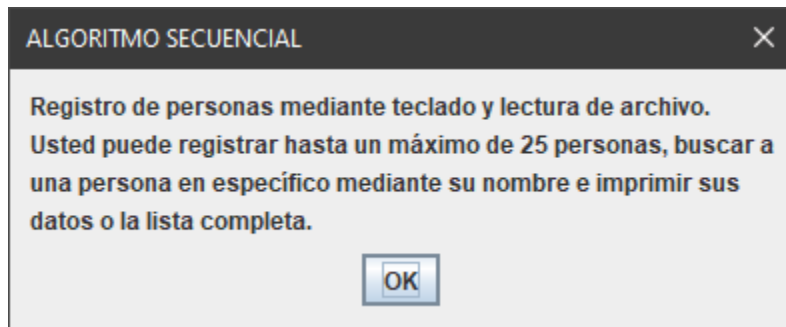
```

        case 3 -> mn.imprimir();
    } sw = mn.menu();
}
}
}

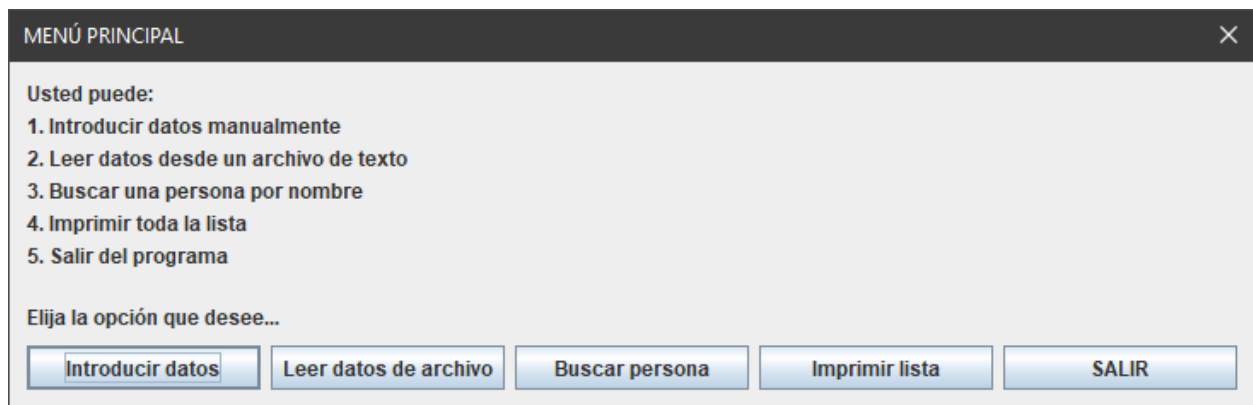
```

7. Capturas de corrida

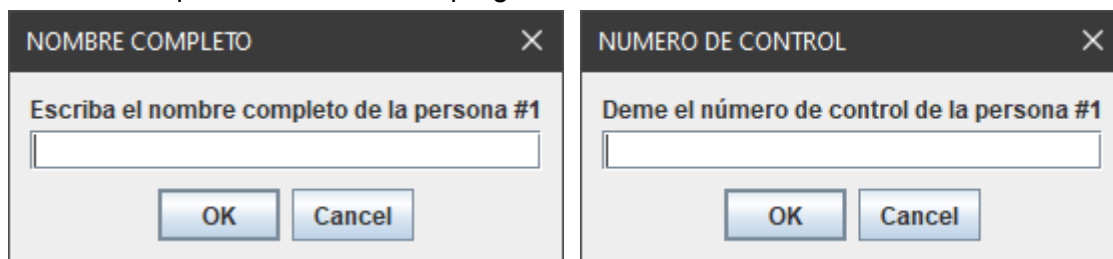
1. META: Imprime la meta del programa.



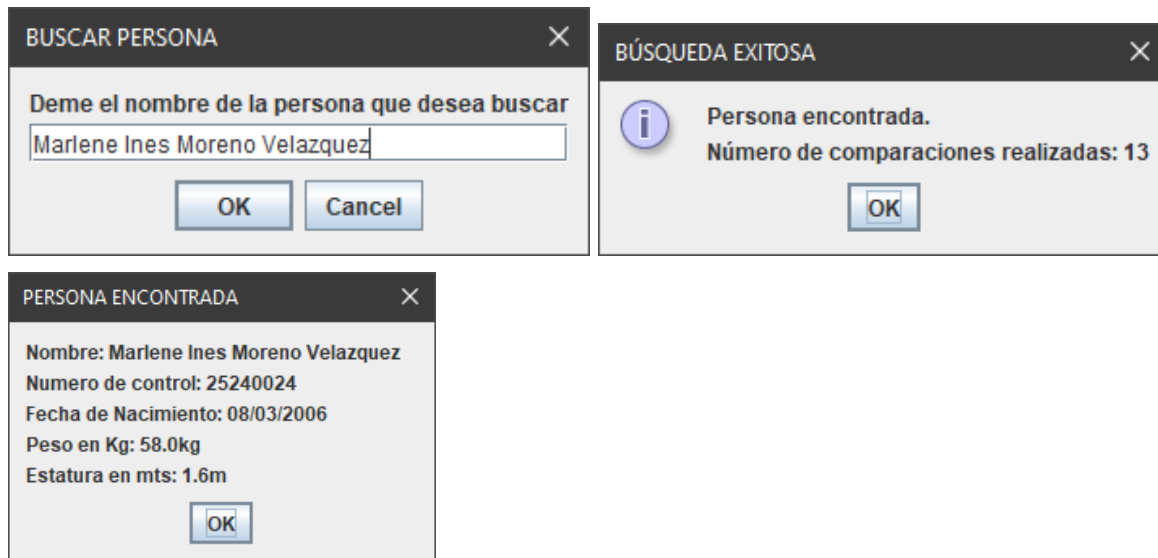
2. NAVEGABILIDAD: Despliega un mensaje informativo con las opciones disponibles y los respectivos botones de cada opción.



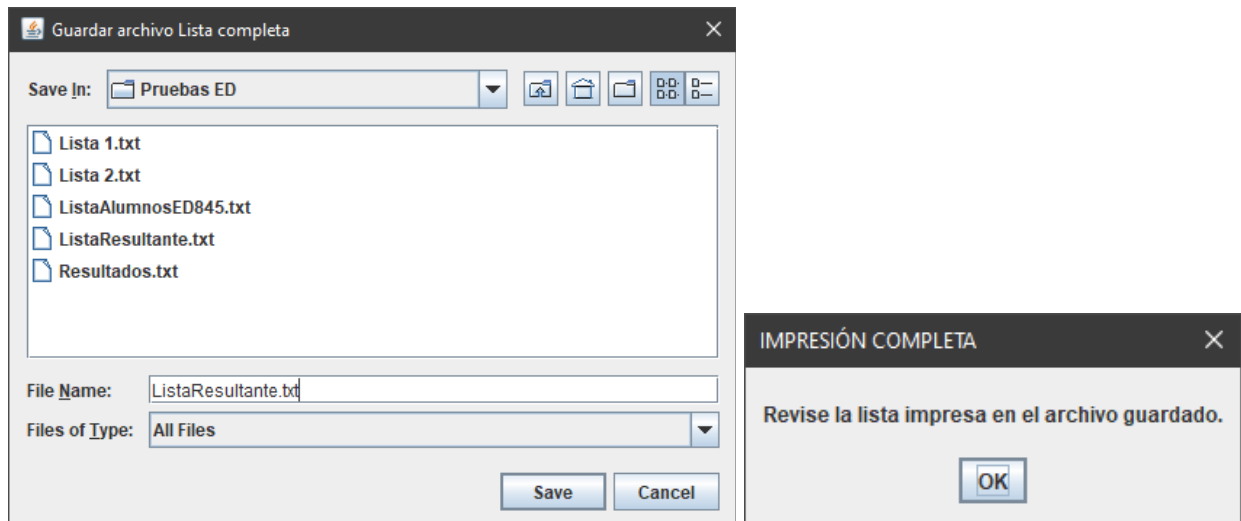
3. DATA (Entrada manual o mediante teclado de datos): Se ejecutan varias ventanas pidiendo cada dato requerido al usuario. El programa tiene sistema de validación de datos.



6. BÚSQUEDA: Ejecuta una ventana que permite al usuario buscar la ficha de una persona mediante su nombre completo. Posteriormente reportará el número de comparaciones realizadas y la ficha de la persona buscada.



7. RESULTADOS: Ejecuta una ventana que permite al usuario guardar la lista ordenada con las fichas correspondientes a cada persona registrada en el programa.



8. FINALIZACIÓN: Si el usuario selecciona la opción “Salir” o da clic a la X de la ventana, el programa se cerrará luego de mostrar el mensaje de cierre.

